

Open-LLM-VTuber on NVIDIA DGX Spark

This guide documents the complete installation and operation of Open-LLM-VTuber on a NVIDIA DGX Spark (GB10 Grace-Blackwell, 128 GB unified memory). All AI models run fully locally on the same GPU simultaneously — no cloud services required.

Architecture

The DGX Spark runs four AI services concurrently: llama-server (llama.cpp with CUDA) serving the HauhauCS Qwen3.6-35B-A3B uncensored LLM on port 8080, faster-whisper large-v3-turbo (CUDA) for speech recognition, MeloTTS (CUDA) for speech synthesis, and Silero VAD (CPU) for voice activity detection. The Open-LLM-VTuber FastAPI server binds port 12393 and connects to llama-server as an OpenAI-compatible backend.

Unified Memory Budget

Component	Memory
LLM — Qwen3.6-35B-A3B Q5_K_P GGUF	~28 GB
ASR — faster-whisper large-v3-turbo	~3 GB
TTS — MeloTTS EN-Default	~2 GB
VAD — Silero VAD	~0.1 GB
KV cache — 128K context at q8_0	~8 GB
OS overhead	~8 GB
TOTAL	~49 GB (79 GB free of 128 GB)

Model Background — HauhauCS Uncensored

HauhauCS (<https://huggingface.co/HauhauCS>) is a Hugging Face creator specialising in lossless uncensored models. Their Qwen3.6 releases score 0/465 refusals on industry refusal benchmarks — the model is completely unlocked and will never refuse a prompt.

Key variants: The Aggressive variant strips self-reasoning and delivers raw answers directly (recommended for creative conversation and roleplay). The Balanced variant keeps reasoning inline and is more stable for long agentic tool chains and coding work.

Why Qwen3.6-35B-A3B? It is a Mixture-of-Experts architecture with 35B total parameters but only ~3B active per forward pass. This means generation speed stays practical for real-time voice conversation while the model knowledge base matches a 35B class model. The Q5_K_P quant (~28 GB) uses HauhauCS custom K_P (Perfect) quantisation which preserves quality 1-2 levels above the base quant at only ~5-15% larger file size.

Disk and Bandwidth Requirements

Asset	Size	Notes
LLM GGUF	~28 GB download	One-time; stored in models/
faster-whisper model	~3 GB download	Auto-downloaded on first boot
MeloTTS voice	~0.5 GB download	Auto-downloaded on first use

Asset	Size	Notes
Total install	~50 GB disk	Code + venv + cached models

Quick Install — Automated Script

Clone the fork and run the setup script. This handles system dependencies, llama.cpp from source (CUDA-enabled), Python dependencies, LLM download, ASR pre-cache, config deployment, and the Hermes verify manifest.

```
git clone https://github.com/subject2change/Open-LLM-VTuber.git
```

```
cd Open-LLM-VTuber
```

```
bash examples/dgx-spark/setup.sh
```

Manual Install — Step by Step

1. System Dependencies

```
sudo apt-get update && sudo apt-get install -y build-essential cmake curl git make libopenblas-dev libomp-dev python3-dev
```

2. Install uv Package Manager

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

3. Python Dependencies

```
cd Open-LLM-VTuber && uv sync
```

```
uv pip install faster-whisper melo-tts==0.1.2 pytest
```

faster-whisper provides GPU-accelerated speech-to-text via CTranslate2. MeloTTS provides local GPU-accelerated text-to-speech. pytest supports the hermes verify test phase.

4. Build llama.cpp from Source

```
git clone --depth 1 https://github.com/ggml-org/llama.cpp build/llama.cpp
```

```
cd build/llama.cpp && cmake -B build -DGGML_CUDA=ON . && cmake --build build --config Release -j$(nproc) --target llama-server
```

```
sudo cp build/bin/llama-server /usr/local/bin/
```

Building from source is necessary for Blackwell GPU support. The -DGGML_CUDA=ON flag enables full CUDA acceleration.

5. Download the LLM Model

Download the Qwen3.6-35B-A3B Aggressive Q5_K_P GGUF (~28 GB) from Hugging Face:

```
curl -L --fail --retry 3 -C - -o models/filename.gguf https://huggingface.co/HauhauCS/Qwen3.6-35B-A3B-Uncensored-HauhauCS-Aggressive/resolve/main/Qwen3.6-35B-A3B-Uncensored-HauhauCS
```

-Aggressive-Q5_K_P.gguf

6. Deploy Config and Hermes Manifest

```
cp examples/dgx-spark/conf.example.yaml conf.yaml
```

Then create `.hermes/environment.json` with `start=uv run run_server.py, port=12393`. This manifest tells hermes verify to use `run_server.py` on port 12393 rather than the auto-detected `uvicorn main:app` on port 8000.

Running the VTuber

You need two terminal sessions (or tmux).

Terminal 1 — llama.cpp LLM Backend

```
llama-server -m models/Qwen3.6-35B-A3B-Uncensored-HauhauCS-Aggressive-Q5_K_P.gguf
--host 127.0.0.1 --port 8080 -ngl 99 -c 131072 --jinja --mlock -fa on --cache-type-k q8_0
--cache-type-v q8_0 --chat-template-kwarg 'enable_thinking: false'
```

Key flags: `-ngl 99` offloads all layers to GPU. `-c 131072` sets 128K context matching the model native support. `--jinja` enables proper chat template handling. `-fa on` enables flash attention. `--cache-type-k/v q8_0` uses 8-bit KV cache to save memory. `--chat-template-kwarg` disables thinking mode for faster voice chat responses.

Terminal 2 — Open-LLM-VTuber Server

```
cd Open-LLM-VTuber && uv run run_server.py --verbose
```

Wait for 'Starting server on localhost:12393', then open `http://localhost:12393`. Click the microphone icon to start speaking — the LLM responds through the Live2D avatar with voice.

Important: First-Boot ASR/TTS Downloads

On the very first run, the ASR and TTS backends auto-download their models. `faster-whisper` downloads `large-v3-turbo` (~3 GB) into `models/whisper/`. `MeloTTS` downloads the `EN-Default` voice (~0.5 GB) on first use. These downloads happen during server startup and can take several minutes depending on your internet speed. Subsequent boots are instant. The `setup.sh` script attempts to pre-cache `faster-whisper` during installation, but if this fails the download will happen on first VTuber boot.

Verification — hermes verify

After the first manual boot (to cache ASR/TTS downloads), verify the full stack: `hermes verify --json --ready-timeout 120`

This runs `uv sync`, `pytest`, starts the server, polls for readiness at port 12393, and tears down. The `--ready-timeout 120` accounts for the server startup time.

Model Selection Guide

LLM Options — All Uncensored HauhauCS

Model	Quant	Size	Notes
Qwen3.6-35B-A3B Aggressive (default)	Q5_K_P	~28 GB	Best quality/latency for 128 GB
Qwen3.6-27B Balanced	Q6_K_P	~23 GB	Dense 27B; stable for agentic chains
Qwen3.6-35B-A3B Aggressive	Q4_K_M	~21 GB	Lighter, faster startup
Gemma4-12B QAT Balanced + MTP	Q4_K_M	~7.4 GB	~60% faster via speculative decoding
Qwen3.6-35B-A3B Aggressive	Q8_K_P	~44 GB	Maximum quality

TTS Options

Model	GPU	Quality	Notes
MeloTTS (default)	~2 GB	4/5	Fully local, GPU-accelerated
edge-tts	0 GB	5/5	Zero GPU, needs internet, instant startup
Piper TTS	0 GB (CPU)	3/5	Local, CPU-only, many voices
Sherpa-ONNX TTS	~1 GB	3/5	Local, GPU-accelerated

ASR Options

Model	VRAM	Latency	Accuracy
faster-whisper large-v3-turbo (default)	~3 GB	~100ms	5/5
faster-whisper medium	~1.5 GB	~50ms	4/5
sherpa-onnx sense-voice	~1 GB (CPU)	~50ms	4/5
whisper.cpp small	~1 GB	~80ms	3/5

Tuning for Lower VRAM

For GPUs with less than 32 GB VRAM, switch to edge-tts (zero GPU), use a smaller LLM (Gemma4-12B at ~7.4 GB), and use faster-whisper medium with int8 compute. Also reduce llama.cpp context to -c 4096.

Troubleshooting

CUDA Out of Memory

1. Use a smaller LLM (Gemma4-12B Q4_K_M ~7.4 GB, or Qwen3.6-35B-A3B Q4_K_M ~21 GB).
2. Switch to edge-tts (zero GPU).
3. Use faster-whisper medium with compute_type int8.
4. Reduce context to -c 4096.
5. Reduce cache-type-k/v to q4_0.

Connection Refused on 127.0.0.1:8080

Ensure llama-server is running in Terminal 1 before starting the VTuber. Verify with: curl -s http://127.0.0.1:8080/v1/models | head -5

Server Fails to Start / Times Out on First Boot

On first boot, the ASR backend downloads its model (~3 GB for faster-whisper, ~1 GB for sherpa-onnx) during startup. This can take several minutes. The server becomes ready once

downloads complete. If using hermes verify, pass --ready-timeout 300.

hermes verify Detects Wrong Port

The auto-detector assumes uvicorn main:app --port 8000 for FastAPI projects. Install the manifest at .hermes/environment.json (included in setup.sh step 8) to override this.

Audio Not Working

Check microphone: pactl list sources short. Verify with: arecord -d 3 test.wav && aplay test.wav. Set the correct input device in your browser.

Frontend Shows Not Found

Initialize the frontend submodule: git submodule update --init --recursive

Dependency Inventory

Package	Purpose	Size
llama-server (from source)	LLM inference, CUDA-enabled	~50 MB binary
faster-whisper	Speech-to-text via CTranslate2/CUDA	~20 MB pip
melo-tts==0.1.2	GPU-accelerated text-to-speech	~30 MB pip
pytest	Test runner for hermes verify	~2 MB pip
HauhauCS Qwen3.6 GGUF	Uncensored LLM	~28 GB download
faster-whisper large-v3-turbo	ASR model (auto-download)	~3 GB download
MeloTTS EN-Default	TTS voice (auto-download)	~0.5 GB download

Key Design Decisions

Why openai_compatible_llm instead of llama_cpp_llm? The built-in llama_cpp_llm provider loads the GGUF directly in-process via llama-cpp-python and exposes very few configuration options — no n_gpu_layers, no context size control, no flash attention. Using llama-server as a standalone process gives full control over GPU offloading, context size, the chat template, and KV cache parameters. It also keeps the model loaded between VTuber restarts.

Why Q5_K_P instead of Q4_K_M? The K_P (Perfect) quant preserves quality 1-2 levels above its base quant level at only ~5-15% larger file size. At Q5_K_P (~28 GB), there is still ~79 GB free on a 128 GB system, so the quality improvement is effectively free.

License

Open-LLM-VTuber: MIT license. llama.cpp: MIT license. Model files (HauhauCS releases): review the model card and license on the Hugging Face repo page before redistribution.

Repository

Fork: <https://github.com/subject2change/Open-LLM-VTuber>

Branch: dgx-spark-setup (PR #1)

Setup files: `examples/dgx-spark/conf.example.yaml`, `examples/dgx-spark/setup.sh`

Model source: <https://huggingface.co/HauhauCS>

PR: <https://github.com/subject2change/Open-LLM-VTuber/pull/1>